10G EMAC

Ethernet Design & Architecture Guide

Verilog-Based 10G Gigabit Media Access Controller

An IP Design Specification · Architecture Guide · Learning Handbook

Prepared by: Rahul | VLSI Design Verification Engineer

June 2026

Table of Contents

Chapter 1	Introduction & Ethernet Fundamentals	3
Chapter 2	OSI Model & Data Link Layer	4
Chapter 3	Ethernet Frame Format	5
Chapter 4	Complete 10G EMAC Architecture	6
Chapter 5	TX Enqueue & TX Dequeue	8
Chapter 6	Pin Description & Error Handling	10
Chapter 7	Internal Building Blocks	12
Chapter 8	TX Dequeue State Machine	15
Chapter 9	Prerequisites & Foundational Concepts	17
Chapter 10	Conclusion	18

Chapter 1 — Introduction & Ethernet Fundamentals

1.1 What is the 10G EMAC Project?

The **10G EMAC (10-Gigabit Ethernet Media Access Controller)** is a high-speed digital IP core implemented in Verilog/SystemVerilog. It governs how raw data is packaged into Ethernet frames, transmitted over a 10 Gbps physical link, and reliably received at the other end — all at wire speed with no CPU intervention on the critical data path.

1.2 Why 10G Ethernet?

Legacy protocols such as UART, SPI, and I²C operate at kilobits-per-second to a few megabits-per-second — entirely inadequate for high-bandwidth applications. PCIe is faster but point-to-point and unsuited to packet-switched networks. 10G Ethernet provides a standardised, widely deployed, packet-based fabric capable of sustaining 10 Gbps throughput across data-centre fabrics, storage networks, and telecom line cards.

1.3 Frequency & Bandwidth Calculation

The XGMII interface transfers 64 bits in parallel each clock cycle. To sustain 10 Gbps the required clock frequency is:

Parameter	Value
Line Rate	10,000 Mbps (10 Gbps)
XGMII Bus Width	64 bits
Required Clock	10,000 ÷ 64 = 156.25 MHz
Verification	156.25 MHz × 64 bits = 10,000 Mbps ✓

1.4 Protocol Comparison

Protocol	Speed	Topology	Use Case
UART	~1 Mbps	Point-to-Point	Debug / console
SPI	~50 Mbps	Master-Slave	Sensor interfaces
I²C	~3.4 Mbps	Multi-drop	Config registers
PCIe	~128 Gbps (x16)	Point-to-Point	CPU↔GPU, storage
10G Ethernet	10 Gbps	Switched network	Data-centre, storage, telecom

Chapter 2 — OSI Model & Data Link Layer

2.1 The Core Problem

When raw bits travel over a cable the receiver faces four fundamental questions: **(1)** Where does data begin? **(2)** Where does it end? **(3)** Is any bit corrupted? **(4)** Who is the intended recipient? The **Data Link Layer (Layer 2)** of the OSI model exists precisely to answer these questions by converting raw bit-streams into structured *frames*.

2.2 The Seven OSI Layers

Layer	Name	Function	Address Unit
7	Application	Human–computer interaction; network services (HTTP, FTP, DNS)	—
6	Presentation	Data format, encryption, compression	—
5	Session	Maintain connections, control ports and sessions	—
4	Transport	End-to-end delivery via TCP (reliable) or UDP (fast)	Port
3	Network	Routing via IP addressing, packet forwarding	IP Address
2	Data Link	Frame structure, MAC addressing, CRC, access control (our project)	MAC Address
1	Physical	Raw bit-stream; electrical/optical signalling	Bit / Symbol

2.3 Data Link Sub-layers

Sub-layer	Full Name	Responsibilities
MAC	Media Access Control	Ethernet framing · MAC addresses · CRC · Access control · Frame TX/RX
LLC	Logical Link Control	Communication with upper layers · Protocol identification · Multiplexing

2.4 MAC vs PHY Boundary

The **MAC** (Media Access Controller) handles everything from frame assembly down to the XGMII bus — this is the digital domain our project covers. The **PHY** (Physical Layer Device) sits below the XGMII interface and converts digital symbols into electrical or optical signals on the wire. The XGMII (10-Gigabit Media-Independent Interface) is the standardised handoff point carrying 64-bit data plus 8-bit control on each clock edge at 156.25 MHz.

Chapter 3 — Ethernet Frame Format

3.1 Frame Fields

Field	Size	Value / Range	Description
Preamble	7 bytes	0xAA…AA (10101010×7)	Bit-clock synchronisation; alternating 1s and 0s
SFD	1 byte	0xD5 (10110101)	Start Frame Delimiter — marks end of preamble
Destination MAC	6 bytes	Variable	Unicast, Multicast, or Broadcast (FF:FF:FF:FF:FF:FF)
Source MAC	6 bytes	Variable	Sender hardware address
Length / EtherType	2 bytes	46–1500 or type code	Payload length OR L3 protocol identifier
Payload	46–1500 B	User data	Minimum 46 B ensures total frame ≥ 64 B
FCS / CRC	4 bytes	CRC-32 result	Frame Check Sequence for error detection

3.2 Frame Size Boundaries

Category	Size	Status	Notes
Runt Frame	< 64 bytes	■ Invalid	Collision fragment or truncated frame; must be discarded
Standard Frame	64–1518 bytes	■ Valid	Normal Ethernet frame; MAC + LLC + Payload + FCS
Jumbo Frame	> 1518 bytes	■ Optional	Requires mutual negotiation; up to 9000 B common in SANs

3.3 Why Minimum Payload is 46 Bytes

The minimum Ethernet frame is **64 bytes** (required for collision detection in half-duplex CSMA/CD). Subtracting the fixed overhead — 6 (DA) + 6 (SA) + 2 (Type) + 4 (FCS) = 18 bytes — leaves exactly **46 bytes** as the minimum payload. If actual data is shorter, the MAC inserts *padding bytes* (0x00) to reach 46 B.

3.4 MAC Address Types

Type	Pattern	Description
Unicast	LSB of first byte = 0	One transmitter → one specific receiver
Multicast	LSB of first byte = 1	One transmitter → group of receivers
Broadcast	FF:FF:FF:FF:FF:FF	One transmitter → all devices on segment

Chapter 4 — Complete 10G EMAC Architecture

4.1 Top-Level Overview

The 10GE MAC core is divided into a **TX Path** and a symmetric **RX Path**, unified by a **Wishbone CPU interface**, a shared **Fault State Machine**, and the **XGMII** boundary toward the PHY. The simplified architecture diagram below shows the major blocks and their interconnections.

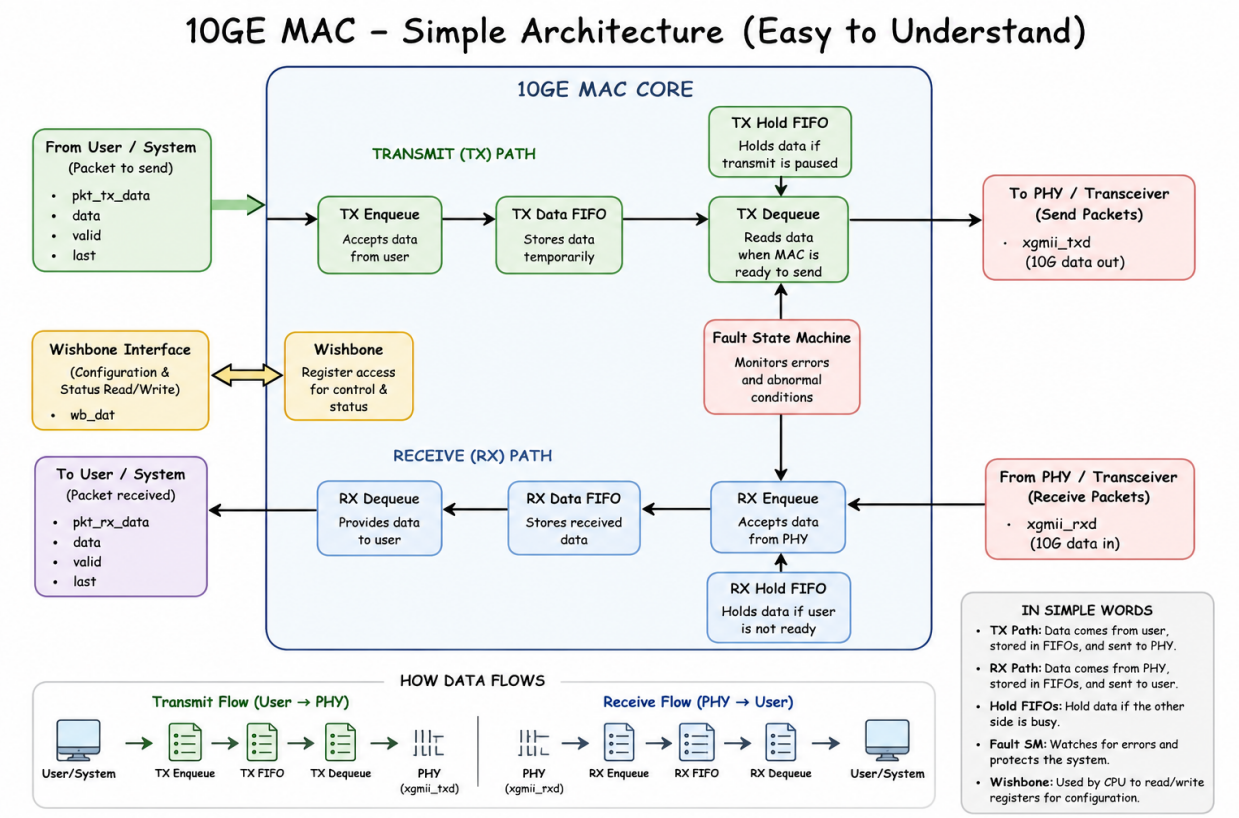


Figure 4-1 | 10GE MAC — Simplified Architecture Overview

4.2 TX Data Path

User logic drives packet signals (*pkt_tx_data, pkt_tx_val, pkt_tx_sop, pkt_tx_eop, pkt_tx_mod*) into **TX Enqueue**, which writes frames into the **TX Data FIFO**. **TX Dequeue** reads from the FIFO, prepends preamble/SFD, appends CRC-32, and serialises the formatted frame onto the XGMII bus (*xgmii_txd / xgmii_txc*) toward the PHY. The **TX Hold FIFO** absorbs short bursts when the MAC must pause transmission.

4.3 RX Data Path

Incoming XGMII data is latched by **RX Enqueue**, which strips preamble, SFD, and IFG, validates the CRC-32, and writes the payload into the **RX Data FIFO**. **RX Dequeue** presents the clean payload to user logic with SOP/EOP/MOD framing. An **RX Hold FIFO** provides back-pressure buffering when the downstream user is not yet ready.

4.4 Loopback Mechanism

For internal functional verification, the TX XGMII outputs are wired directly back to the RX XGMII inputs:

```
assign xgmii_rxd = xgmii_txd; // 64-bit data loopback
assign xgmii_rxc = xgmii_txc; // 8-bit control loopback
```

This allows self-test of the complete TX→RX path without an external PHY or cable.

4.5 Wishbone Interface

The **Wishbone** bus is an open-standard on-chip interconnect used to interface the high-speed MAC logic with a low-speed CPU/microcontroller. The MAC exposes **4 registers**:

Register	Type	Purpose
ctrl_tx_enable	Configure	Enable / disable the TX path; set loopback mode
status_crc_error	Status	Asserts if a received frame fails CRC-32 check
status_fragment_error	Status	Asserts on fragment error (SOP without prior EOP)
status_pause_frame	Status	Asserts when a PAUSE frame is received from peer

4.6 Full Block Diagram

The detailed RTL block diagram below shows all signal interconnections including TX/RX FIFOs, Wishbone, Fault SM, and the XGMII boundary.

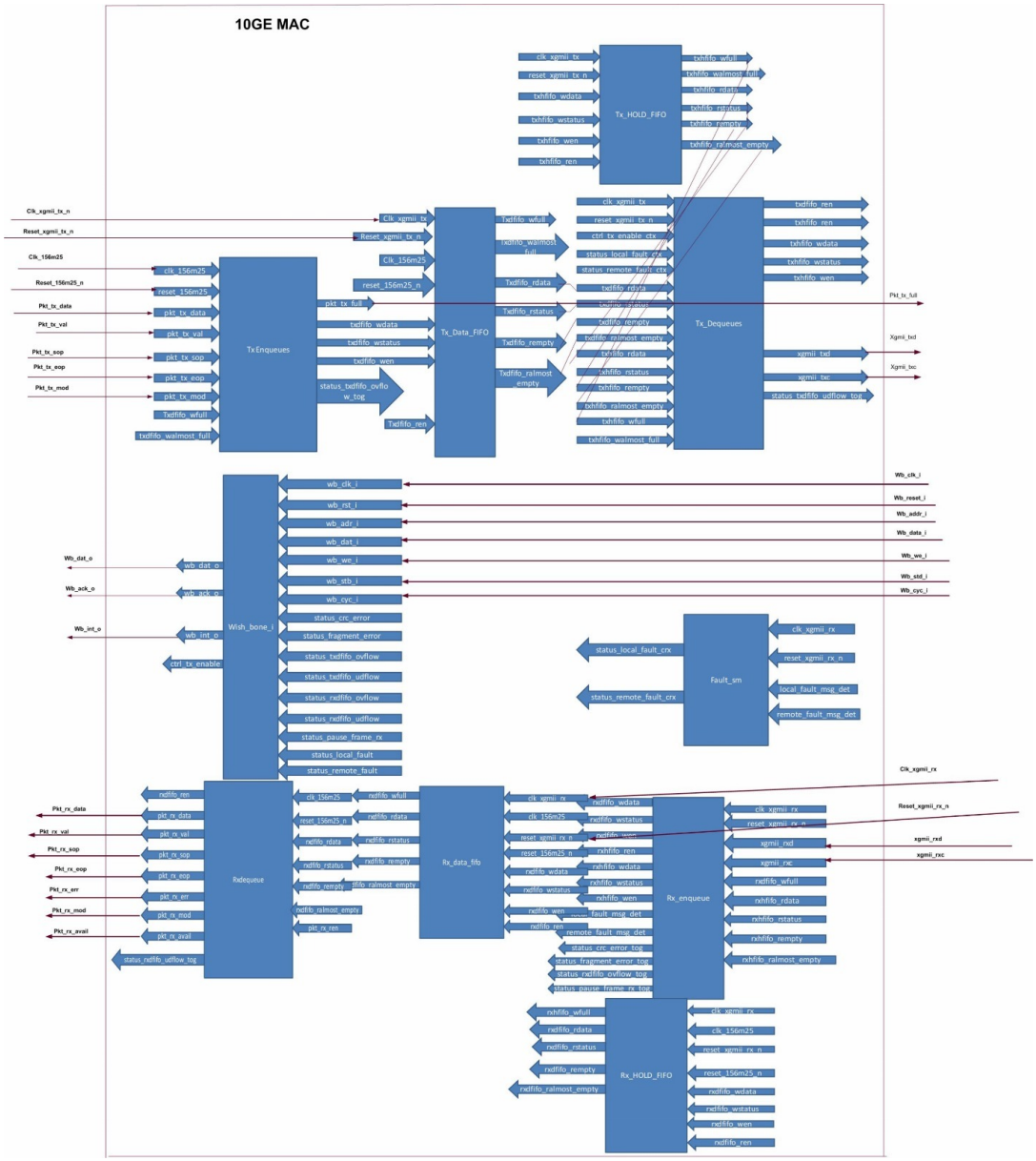


Figure 4-2 | 10GE MAC — Detailed RTL Block Diagram (all signal buses)

4.7 Fault State Machine

The **Fault SM** monitors both **Local Faults** (the receiver cannot decode data the TX has sent) and **Remote Faults** (the peer reports it cannot receive). The FSM uses a threshold rule to avoid false triggers:

- 1 column = 32 bits of XGMII data
- The FSM observes a window of **128 columns** (= 4096 bits)
- A fault is flagged only when **more than 4 fault patterns** appear inside the window
- Isolated glitches (1–4 occurrences) are silently ignored
- Local fault output: **9C sequence**; Remote fault output: **IDLE**

Chapter 5 — TX Enqueue & TX Dequeue

5.1 TX Data Flow Overview

Stage	Block	Description
①	TX Enqueue	Accepts user data (pkt_tx_data, val, sop, eop, mod); writes to TX Data FIFO
②	TX Data FIFO	72-bit wide async FIFO buffers complete frames
③	TX Hold FIFO	Secondary FIFO holds data when TX is paused or inserting IFG
④	TX Dequeue	Reads FIFO, prepends preamble/SFD, appends CRC-32, inserts IFG
⑤	RC Layer	Reconciliation sublayer converts MAC frame to XGMII format
⑥	PHY	Physical layer receives XGMII → drives the 10G serial link

5.2 TX Enqueue

TX Enqueue is the user-facing front-end. It monitors *pkt_tx_sop* to detect frame starts and *pkt_tx_eop* for frame ends. On every valid cycle (*pkt_tx_val=1*) it writes 64-bit data plus a byte-enable mask into the TX Data FIFO. It also detects **Fragment Errors** — when a new SOP arrives before the current frame's EOP — and raises *status_fragment_error* via the Wishbone status register.

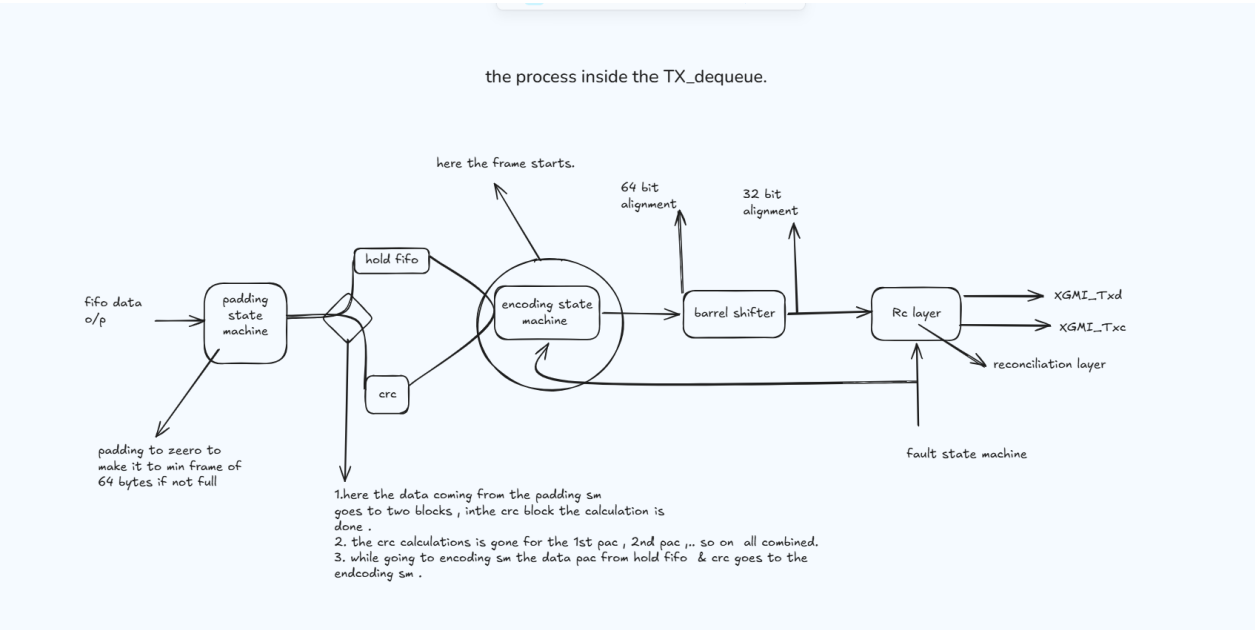


Figure 5-1 | TX Dequeue — Internal Process Flow (Padding SM → Hold FIFO / CRC → Encoding SM → Barrel Shifter → RC Layer)

5.3 TX Dequeue Sub-blocks

5.3.1 Encoding State Machine

The Encoding SM constructs the on-wire byte sequence. Fixed control byte values used:

Signal	Fixed Value	Purpose
Start	0xFD	XGMII Start control character (lane 0)

Signal	Fixed Value	Purpose
Preamble	0x555555…	Seven preamble bytes after the Start character
SFD	0xD5	Start Frame Delimiter (final preamble byte)
Terminate	0xFD	XGMII Terminate control character
IFG	0x07070707…	Inter-Frame Gap idle bytes (96 bits minimum)

5.3.2 Barrel Shifter

A barrel shifter is a combinational circuit that rotates a data word by a *variable* number of bit/byte positions in a **single clock cycle** — no iterative shifting loops needed. In the TX path it is required because:

- Ethernet frames must start on **Lane 0** or **Lane 4** of the XGMII bus
- Lane 4 alignment allows 32-bit datapath partitioning simplifying CRC logic
- When EOP falls mid-word the barrel shifter aligns the next frame's start to Lane 0/4
- It reads 32 bits from the *current* packet and 32 bits from the *next* packet to complete a 64-bit XGMII word
- Remaining IFG bytes are truncated (minimum IFG compliance is still maintained)

5.3.3 Inter-Frame Gap (IFG)

Between consecutive frames a mandatory silence gap of **12 bytes (96 bits)** must be observed. These are filled with the XGMII idle byte **0x07**. The standard IFG byte pattern output is 07 07 07 07 07 07 07 07 07 07 07 07.

5.3.4 Reconciliation (RC) Layer

The RC layer is the final stage before the PHY. It converts internal MAC signals into **XGMII format** with two output buses:

Output Bus	Width	Content
XGMII_Txd	64 bits	Data: Start + Preamble + SFD + Payload + CRC + Terminate + Idle
XGMII_Txc	8 bits	Control qualifier per byte lane: 1 = control, 0 = data

The Txc signal goes HIGH on bytes that carry control characters (Start, Terminate, Idle, Error). When a **Local Fault** is active the RC layer outputs the **0x9C sequence** on Txd. A **Remote Fault** causes the RC layer to output continuous **IDLE (0x07)** characters.

5.4 RX Enqueue — Internal Process

The RX Enqueue block is the receive-side mirror of TX Dequeue. Incoming XGMII data passes through the RC Layer → Barrel Shifter → Decoding SM, which strips all control characters (Start, Preamble, SFD, Terminate, Idle) and recovers the raw payload. The CRC computed over the received payload is compared against the transmitted FCS appended to the frame — a mismatch raises an error to the Wishbone status register.

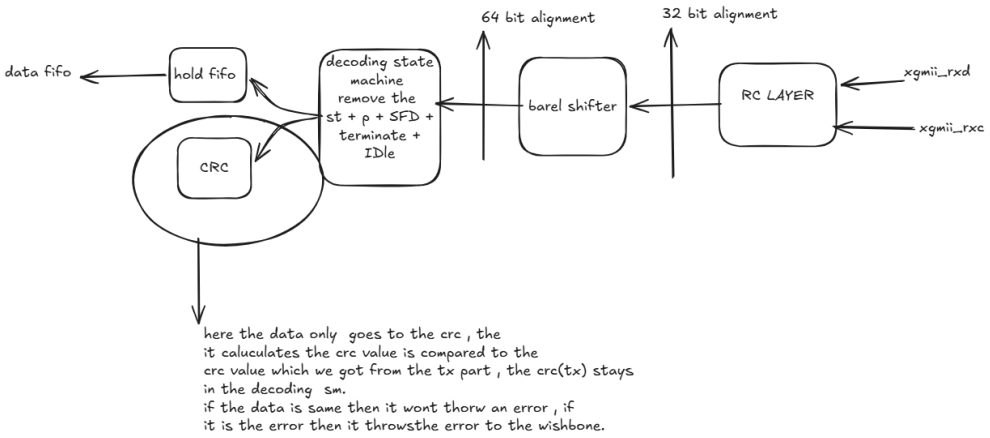


Figure 5-2 | RX Enqueue — Internal Process Flow (RC Layer → Barrel Shifter → Decoding SM / CRC → Hold FIFO → Data FIFO)

CRC verification flow: The Decoding SM passes the stripped payload data to the CRC block, which computes a fresh CRC-32. The TX-side CRC (transmitted as the last 4 bytes of the frame) is retained inside the Decoding SM. If the two values match, the frame is forwarded to the RX Data FIFO. If they differ, *status_crc_error* is asserted via Wishbone and the frame is discarded.

Chapter 6 — Pin Description & Error Handling

6.1 Clock & Reset Signals

Signal	Dir	Width	Description
Clk_xgmii_tx	In	1	156.25 MHz TX clock (XGMII domain)
Reset_xgmii_tx_n	In	1	Active-low async reset for TX XGMII
Clk_xgmii_rx	In	1	156.25 MHz RX clock (XGMII domain)
Reset_xgmii_rx_n	In	1	Active-low async reset for RX XGMII
Clk_156m25	In	1	156.25 MHz core clock
Reset_156m25_n	In	1	Active-low reset for core logic

6.2 TX User Interface

Signal	Dir	Width	Description
Pkt_tx_data	In	64	Packet payload data (big-endian)
Pkt_tx_val	In	1	Data valid qualifier
Pkt_tx_sop	In	1	Start of Packet pulse
Pkt_tx_eop	In	1	End of Packet pulse
Pkt_tx_mod	In	3	Number of valid bytes in the last word (0=all 8 valid)
Pkt_tx_full	Out	1	TX FIFO full — user must pause
Txdfifo_wfull	Out	1	TX data FIFO write-full flag
Txdfifo_walmost_full	Out	1	TX data FIFO almost-full flag

6.3 RX User Interface

Signal	Dir	Width	Description
Pkt_rx_data	Out	64	Received payload data
Pkt_rx_val	Out	1	Data valid qualifier
Pkt_rx_sop	Out	1	Start of Packet
Pkt_rx_eop	Out	1	End of Packet
Pkt_rx_err	Out	1	Error in received frame (CRC, fragment, etc.)
Pkt_rx_mod	Out	3	Valid bytes in last word
Pkt_rx_avail	Out	1	Data available in RX FIFO

6.4 XGMII Interface (MAC ↔ PHY)

Signal	Dir	Width	Description
xgmii_txd	Out	64	64-bit TX data to PHY; updates every 156.25 MHz cycle
xgmii_txc	Out	8	TX control per byte lane (1=control character)
xgmii_rxd	In	64	64-bit RX data from PHY

Signal	Dir	Width	Description
xgmii_rxc	In	8	RX control per byte lane

6.5 Wishbone Interface

Signal	Dir	Width	Description
Wb_clk_i	In	1	Wishbone bus clock
Wb_rst_i	In	1	Wishbone synchronous reset
Wb_adr_i	In	8	Register address
Wb_dat_i	In	32	Write data from CPU
Wb_dat_o	Out	32	Read data to CPU
Wb_we_i	In	1	Write enable
Wb_stb_i	In	1	Strobe — cycle valid qualifier
Wb_cyc_i	In	1	Bus cycle active
Wb_ack_o	Out	1	Acknowledge — transaction complete
Wb_int_o	Out	1	Interrupt to CPU (fault or error event)

6.6 Error Types

Error	Trigger Condition	Effect
Fragment Error	New SOP arrives while a previous SOP has not been closed by an EOP	pkt_rx_err=1, packet discarded; status register set
Pause-Frame Error	A PAUSE frame is received from the link partner	TX path stalls for the duration specified in the pause frame set via Wishbone
CRC Error	FCS computed ≠ FCS received in the frame	Frame dropped; status_crc_error asserted
Local Fault	RX side cannot decode data from the TX partner	RC layer inserts 0x9C sequence; Fault SM triggered
Remote Fault	Link partner reports it cannot receive	RC layer inserts IDLE continuously until fault clears

Chapter 7 — Internal Building Blocks

7.1 FIFO Architecture

7.1.1 Hierarchy

The FIFO hierarchy is:

```
tx_data_fifo (top wrapper)
■■■ generic_fifo
■■■ generic_fifo_ctrl (read/write pointer logic, flags)
■■■ generic_mem_medium (TX data FIFO — main payload storage)
■■■ generic_mem_small (TX hold FIFO — pause/EOP buffering)
```

7.1.2 Why a Shift-Register Model?

Traditional circular-buffer FIFOs use a single shared memory array with read and write pointers. When two simultaneous operations need *different* memory locations (e.g., data FIFO write + hold FIFO read) they conflict on the shared bus. This design instead uses a **shift-register FIFO** — each location has its own register cell — eliminating structural hazards and enabling simultaneous independent read and write operations.

7.1.3 Asynchronous Clock Domain Crossing

Because the write clock (user logic domain) and read clock (XGMII domain) are independent, the FIFO controller performs **binary-to-Gray code conversion** on the write pointer before synchronising it into the read clock domain (and vice versa). Gray code ensures that only one bit changes per pointer increment, eliminating multi-bit metastability glitches during CDC synchronisation.

Operation	Domain	Mechanism
Write pointer → RD domain	WR clk → RD clk	Binary → Gray → 2FF synchroniser → Gray → Binary
Read pointer → WR domain	RD clk → WR clk	Binary → Gray → 2FF synchroniser → Gray → Binary
Full flag	WR domain	Derived from synchronised RD pointer
Empty flag	RD domain	Derived from synchronised WR pointer

7.2 CRC-32 Calculation Engine

7.2.1 Step-by-Step Algorithm

The CRC-32 algorithm used in Ethernet is defined by IEEE 802.3. The implementation follows five precise steps:

Step	Operation	Value / Result
1	Original IEEE polynomial	0x04C11DB7
2	Reflect polynomial (bit-reverse)	0xEDB88320
3	Reflect input data (e.g. 0xAB → 0xD5)	Bitwise reverse each input byte
4	Initialise CRC register; XOR first reflected byte	CRC = 0xFFFFFFFF; XOR with reflected data → 0xFFFFFF2A (example)
5	Shift loop per bit: LSB=0 → right-shift; LSB=1 → right-shift, XOR 0xEDB88320	Repeat for all 8 bits, then finalise the CRC-32

7.2.2 Polynomial Reflection Example

```
Original: 0000 0100 1100 0001 0001 1101 1011 0111
= 0 4 C 1 1 D B 7
```

```
Reflected: 1110 1101 1011 1000 1000 0011 0010 0000
= E D B 8 8 3 2 0

Reflected polynomial = 0xEDB88320
```

7.2.3 Data Reflection Example

```
Input byte: 0xAB = 1010 1011
Reflected byte: 0xD5 = 1101 0101
```

7.3 Barrel Shifter Detail

Aspect	Detail
Type	Combinational (no clock); result available in one cycle
Operation	Rotate 64-bit XGMII word by 0, 8, 16, 24, 32, 40, 48, or 56 bits
Control input	3-bit shift amount from TX state machine (lane number)
Key use case	Align next-frame start to Lane 0 after EOP falls on Lane 4–7
Lane 4 advantage	Splits 64-bit word into two 32-bit halves for easier CRC and control insertion

Chapter 8 — TX Dequeue State Machine

8.1 FSM Diagram

The TX Dequeue FSM controls the complete lifecycle of a frame from preamble insertion through CRC append to IFG insertion. The hand-drawn FSM diagram (captured from design notes) is shown below:

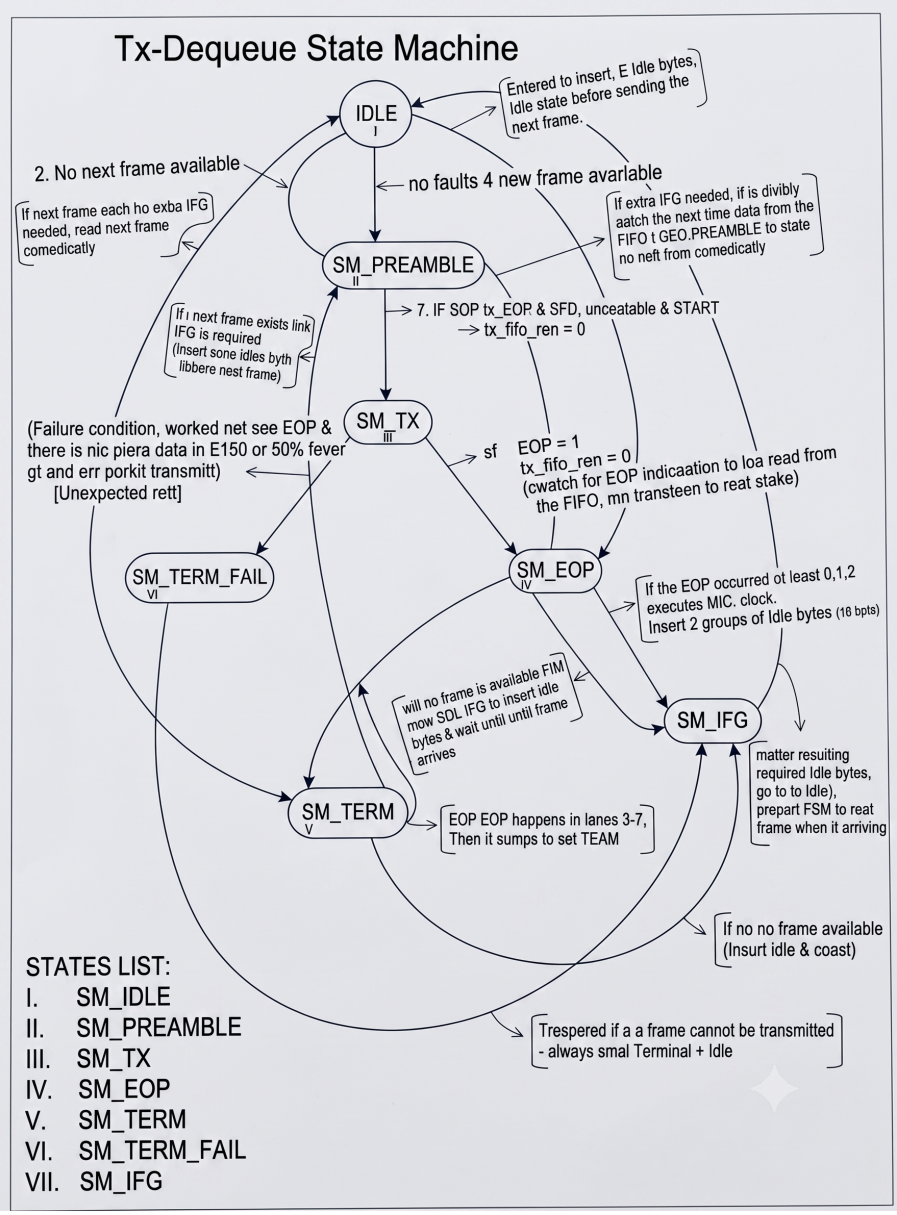


Figure 8-1 | TX Dequeue State Machine — State Transition Diagram

8.2 State Descriptions

State	State #	Entry Condition	Action	Exit Condition
SM_IDLE	I	Reset or after IFG	Output IDLE characters; wait	New frame in FIFO (no active fault)
SM_PREAMBLE	II	SOP detected; no fault	Assert tx_fifo_ren=1; TX preamble bytes	SFD byte output complete

State	State #	Entry Condition	Action	Exit Condition
SM_TX	III	SFD sent	Stream payload bytes from FIFO	EOP=1 detected
SM_EOP	IV	EOP=1; tx_fifo_ren=0	Finalise last data word; compute MCD	Determined EOP lane position
SM_TERM	V	EOP in lanes 3–7	Insert Terminate character	Goto SM_IFG
SM_TERM_FAIL	VI	Unexpected SOP / no EOP / FIFO empty	Assert error flag; flush	Goto SM_IDLE always
SM_IFG	VII	EOP in lanes 0–2 OR from SM_TERM	Insert IDLE (2×8 idle bytes in total)	Next frame ready → SM_PREAMBLE; else SM_IDLE

8.3 Key Transition Rules

- **IDLE → PREAMBLE:** Only when a new frame is available in the FIFO *and* no active fault is detected by the Fault SM.
- **PREAMBLE → TX:** After the Encoding SM has output the full 8-byte preamble (7×0x55 + 0xD5 SFD) with the Start character (0xFD) in Lane 0.
- **TX → EOP:** When *pkt_tx_eop* is asserted and *tx_fifo_ren* is de-asserted.
- **EOP → IFG:** EOP falls on **Lane 0, 1, or 2** — 2 groups of idle bytes (16 bytes total) are inserted before the next preamble.
- **EOP → TERM:** EOP falls on **Lane 3–7** — terminate character inserted; then transitions to SM_IFG.
- **IFG → PREAMBLE:** If the next frame is already in the FIFO, start immediately (back-to-back frames, minimum IFG maintained). Otherwise fall back to SM_IDLE.
- **TERM_FAIL → IDLE:** Always — error recovery forces a return to idle regardless of FIFO state.

Chapter 9 — Prerequisites & Foundational Concepts

Finite State Machines (FSM)

A computational model with a fixed set of states, transition conditions, and outputs. FSMs in digital design are implemented as always blocks with a state register. The TX Dequeue FSM in this project is a Moore machine — outputs depend only on the current state, not the inputs directly.

Shift Registers

A chain of D flip-flops sharing a common clock. Data shifts one position per cycle. Used in this project for preamble generation and as the basic storage cell in the shift-register FIFO architecture.

Clock Domain Crossing (CDC)

When signals cross between logic clocked by different (asynchronous) sources, metastability can occur. The standard mitigation is a 2-flip-flop synchroniser chain. Binary-to-Gray encoding is used for multi-bit pointers to ensure only one bit changes per increment, preventing multi-bit glitch errors.

Parallel vs Serial Throughput

Serial links send one bit per clock per lane. 10G Ethernet uses 4 SerDes lanes at 3.125 Gbps each for a combined 10 Gbps. At the MAC level the XGMII presents 64 parallel bits at 156.25 MHz for easier digital processing.

Full Duplex vs Half Duplex

Half-duplex: only one side transmits at a time (requires CSMA/CD collision detection). Full-duplex: TX and RX operate simultaneously on separate paths (no collisions). 10G Ethernet is always full-duplex — the XGMII carries a separate TX bus and RX bus.

XGMII Interface

The 10-Gigabit Media Independent Interface. It carries 64-bit data (Txd/Rxd) plus 8-bit control qualifiers (Txc/Rxc) on each clock edge at 156.25 MHz. Each control bit qualifies the corresponding byte: 1 = control character, 0 = data.

Chapter 10 — Conclusion

10.1 Architecture Summary

The 10G EMAC project is a complete, production-grade Ethernet MAC IP core implemented in Verilog. It encompasses a **TX path** (enqueue → data FIFO → hold FIFO → dequeue → XGMII) and a symmetric **RX path**, unified by a Wishbone CPU interface for configuration and status, and a Fault SM for link integrity monitoring. The core operates at 156.25 MHz on a 64-bit datapath, sustaining 10 Gbps wire-rate throughput.

10.2 Key Engineering Insights

- The 156.25 MHz clock derives directly from the 10 Gbps ÷ 64-bit datapath constraint
- CRC-32 is computed on reflected data with a reflected polynomial (0xEDB88320) for hardware efficiency
- Barrel shifting aligns frames to Lane 0 / Lane 4 enabling clean 32-bit datapath partitioning
- Gray-code CDC synchronisation prevents multi-bit pointer glitches across the async FIFO boundary
- The Fault SM threshold (>4 faults / 128 columns) prevents false fault declarations from noise
- IFG of 12 bytes ensures receivers can detect frame boundaries even at maximum line rate

10.3 Real-World Applications

Application Area	Use Case
Data Centre Networking	Top-of-rack switches, server NICs, 10GbE fabric interconnects
Telecommunications	Line cards in edge routers and OTN transponders
Storage Networks (SAN)	iSCSI and FCoE over 10G Ethernet for high-throughput storage
FPGA Prototyping	Soft MAC cores for Xilinx/Intel FPGAs in network appliances
Test & Measurement	Packet generators and analysers for network conformance testing

End of Document — 10G EMAC Ethernet Design & Architecture Guide